

CENG443 – Assignment 2 Report

Object Detection Models on Jetson Nano

1. Steps Followed

1) Installation and Build: We opted to build the project from source rather than using the Docker container to minimize storage usage and avoid container runtime overhead on the Jetson Nano. The following commands were used:

```
git clone --recursive --depth=1 https://github.com/dusty-nv/jetson-inference
```

```
mkdir build && cd build
```

```
cmake ../
```

```
make detectnet -j4
```

- **Compilation vs. Model Optimization:** We compiled the C++ source code for the **detectNet** program using **make**. However, the deep learning models themselves (Mobilenet, Inception, PeopleNet) were not "compiled" in this step. Instead, they were optimized at runtime. When running the program for the first time, the system built a **TensorRT engine** (deserialization), optimizing the model layers specifically for the Tegra X1 GPU.

2) Execution: detectNet was executed with SSD-Mobilenet-v2 on a sample image:

```
./detectnet --network=ssd-mobilenet-v2 images/bird_0.jpg images/test/output.jpg
```

3) Profiling: nvprof was used to profile all three models (mobilenet, inception, peoplenet):

```
sudo /usr/local/cuda/bin/nvprof --csv --log-file output_<model>.csv --metrics  
sm_efficiency ./detectnet --network=<model> images/humans_1.jpg  
images/test/humans_output_<model>_1.jpg
```

2. Execution Time Results

The following average inference times were collected from the `detectnet` log files (Total CPU time):

PeopleNet (Fastest): ~620 ms - 700 ms per image.

Observation: Although PeopleNet detects more complex classes (Face + Person), it ran surprisingly fast on our device after the initial setup. The first run took roughly 4.2 seconds due to the engine building process, but subsequent runs were very efficient.

SSD-Inception-v2: ~730 ms - 800 ms per image.

SSD-Mobilenet-v2: ~800 ms - 890 ms per image.

Observation: In our specific test environment, Mobilenet performed slightly slower than the other optimized models.

Reference Photo Analysis:

Standard Models: In the models detect people with standard bounding boxes.

PeopleNet: In `humans\_output\_peoplenet\_1.jpg`, the model provides more detail. It detects both "person" and "face" classes simultaneously with high confidence

3. SM Efficiency Results (Per-Model Analysis)

Using `nvprof`, we analyzed the Multiprocessor (SM) activity to identify which kernel functions utilized the GPU most and least effectively for each model.

A. SSD-Mobilenet-v2

Highest Efficiency: `fusedConvolutionReluKernel` (~99.6%).

Analysis: This kernel performs dense matrix multiplications (convolutions). Since the GPU cores are continuously calculating without waiting for memory, the efficiency is near maximum.

Lowest Efficiency: `gatherTopDetections\_kernel` (~12.6%).

Analysis: This function handles the sorting of detected bounding boxes (Non-Maximum Suppression). Sorting requires logical checks and irregular memory access, causing the GPU cores to stall and wait, resulting in low efficiency.

B. SSD-Inception-v2

Highest Efficiency: `fusedConvolutionReluKernel` (~99.4%).

Analysis: Similar to Mobilenet, the backbone of this model relies heavily on fused convolution and activation layers, which are highly optimized for the GPU.

Lowest Efficiency: `copy\_kernel` (~28.2%) and `gatherTopDetections\_kernel` (~12.7%).

Analysis: Aside from the sorting bottleneck, this model showed low efficiency during memory copy operations. Moving data between memory spaces is bandwidth-limited, leaving the compute units idle.

C. PeopleNet

Highest Efficiency: `cuInt8::nchwToNchhw2` (~99.9%).

Analysis: This model uses a specialized kernel to manage its data layout. The high efficiency indicates that the data reformatting process for this specific model architecture is extremely optimized for the Tegra X1 hardware.

Lowest Efficiency: `generatedNativePointwise` (~79.8%).

Analysis: While still relatively efficient, these pointwise operations (simple element-by-element math) utilize the GPU less intensively compared to the complex convolution kernels.

4. Output Images

- **SSD-Mobilenet-v2:** `humans_output_mobilenet_1.jpg` (Detects persons)



- **SSD-Inception-v2:** `humans_output_inception_1.jpg` (Detects persons)



- **PeopleNet:** `humans_output_peoplenet_1.jpg` (Detects persons + faces)



5. Understanding and Interpretation

This assignment showed us the trade-off between model startup time and inference speed. While PeopleNet required a long initialization time (over 4 seconds) to build its engine, it offered the fastest inference speed and the most detailed detection (faces + people) during actual use. Profiling with ``nvprof`` revealed that the GPU is most efficient when doing pure mathematical calculations (convolutions) and least efficient when performing logic-heavy tasks like sorting detections. Building from source allowed us to manage the limited storage on the Jetson Nano effectively.